

In bfloat16 the batch shape moves the importance ratio: measurement, controls, and what removes it

Guo Cheng — University of Chinese Academy of Sciences

Technical report, September 2026 — github.com/GuoCheng24/batch-logprob-gap

Abstract. In bfloat16 a transformer gives the same token a different log probability depending on how many sequences share its batch. The policy has not moved and the tokens are identical; only the reduction order changed. Across 4 architectures and 8 models (0.16B–7B) the importance ratio $\exp(\log p_{\text{new}} - \log p_{\text{old}})$ that GRPO-style objectives form at their first inner epoch leaves $[0.9, 1.1]$ for **1.8% to 60.0%** of tokens, and for **0.00%** in fp32. Controls separate the causes: repeating the same batch is bit-exact, so it is deterministic; equal-length batches without padding behave like ragged ones, so it is not padding; eight copies of one sequence differ from that sequence alone, so batch size alone suffices; only the MoE changes when a batch is merely regrouped. The noise reaches the objective — which tokens PPO clipping excludes flips by 1.7% to 10.1% — and it enters where the architecture puts it: the last quarter of a GPT-NeoX stack, the first five layers of Qwen2 and Qwen3, so an fp32 `lm_head` removes most of it on Qwen and almost none on pythia. Upcasting the layers with the largest divergence increment beats a fixed tail at equal parameter budget, but scoring in fp16 removes the effect outright at bf16 cost on every Qwen-family model without overflowing up to 7B. A second, unrelated term appears whenever sampling is truncated: vLLM’s processed log-probs are normalised over the kept tokens, and the trainer’s are not, so at `top_p=0.8` the ratio averages 0.896 for an unchanged policy; adding the log kept mass that vLLM can now replay brings it to 1.001. Finally, six ways of computing GRPO’s old log-probs on Qwen2.5-1.5B-Instruct, two seeds, 150 steps of GSM8K, finish within -0.013 to $+0.008$ of each other: at this scale the noise is real and in the ratio, and it does not reach the reward.

1. Setting

The question is the one left open in verl issue #6280 since May 2026: on-policy GRPO runs report a mismatch between `old_log_prob` and `log_prob` for the same tokens under the same weights — is it caused by kernels that are not batch-invariant? The measurement here is deliberately small: one RTX 4090 (SM89), Hugging Face `transformers` forward passes with no inference engine unless stated, teacher-forced rescoring of sampled continuations, and the out-of-band rate — the fraction of scored tokens whose ratio between two scorings leaves $[0.9, 1.1]$, the band a PPO clip of $\epsilon = 0.1$ would treat as “unchanged” — as the single statistic. Every table in this report is regenerated from committed JSON by `scripts/build_tables.py`; nothing is typed by hand, and the same script produced this document’s tables.

2. The effect, and what it is not

model	bf16 b1 vs b8	fp16 b1 vs b8	fp32 b1 vs b8	bf16 b1 rerun	bf16 b1 vs b32
pythia-160m (NeoX 0.16B)	60.03%	22.02%	0.00%	0.00%	59.70%
pythia-410m (NeoX 0.41B)	36.70%	2.49%	0.00%	0.00%	36.65%
Qwen2.5-0.5B	9.87%	0.07%	0.00%	0.00%	10.75%
Granite-3.1-1B-A400M (MoE)	13.69%	0.97%	0.00%	0.00%	13.85%
Agents-A1-4B	1.78%	0.00%	0.00%	0.00%	1.64%
Qwen2.5-1.5B	9.37%	0.00%	0.00%	0.00%	8.87%
Qwen2.5-3B	6.87%	0.00%	0.00%	0.00%	7.17%

model	bf16 b1 vs b8	fp16 b1 vs b8	fp32 b1 vs b8	bf16 b1 rerun	bf16 b1 vs b32
Qwen2.5-7B	5.60%	0.00%	0.00%	0.00%	5.71%

Repeating the *same* batch is bit-exact (column 4), so this is not nondeterminism; it is a deterministic function of the batch shape. fp16, with the same 16 bits, cuts the rate by an order of magnitude on most models and fp32 removes it. Three controls separate the candidate causes:

model	A padded ragged b1v8	B equal-length no pad b1v8	C same seq x8 vs alone	D two groupings of batch 8
pythia-410m (NeoX 0.41B)	36.87%	36.23%	18.75%	0.00%
Qwen2.5-0.5B	9.74%	10.06%	6.25%	0.00%
Granite-3.1-1B-A400M (MoE)	14.09%	14.21%	20.31%	12.60%
Agents-A1-4B	1.78%	3.25%	6.25%	0.00%

Column B kills padding: equal-length sequences with no padding at all behave like ragged padded ones. Column C shows batch size alone is sufficient: eight identical copies of one sequence, scored together, differ from that sequence scored alone. Column D is zero for every dense model and **12.60%** for the MoE, whose expert routing depends on batch composition — matching the thread’s report that the mismatch appeared on a 35B-A3B mixture and not on dense models. Three other ways of changing the kernel path — an explicit 4D mask, sequence packing — move the rate as well and are all 0.00% in fp32:

model	change	bf16	fp32
pythia-410m (NeoX 0.41B)	batch shape [1,L] vs [8,L]	31.84%	0.00%
pythia-410m (NeoX 0.41B)	mask implicit vs explicit 4D	21.35%	0.00%
pythia-410m (NeoX 0.41B)	packing 1/row vs 8/row	32.23%	0.00%
Qwen2.5-0.5B	batch shape [1,L] vs [8,L]	9.18%	0.00%
Qwen2.5-0.5B	mask implicit vs explicit 4D	4.82%	0.00%
Qwen2.5-0.5B	packing 1/row vs 8/row	9.44%	0.00%

The effect survives `eager`, `sdpa` and `flex_attention`, with per-seed spreads well inside the confidence intervals:

model	implementation	out-of-band	95% CI	per-seed	n
pythia-410m (NeoX 0.41B)	eager	53.91%	[52.46, 55.34]	53.71, 54.30, 53.71	4608
pythia-410m (NeoX 0.41B)	sdpa	32.92%	[31.58, 34.29]	31.84, 32.62, 34.31	4608
pythia-410m (NeoX 0.41B)	flex_attention	33.29%	[31.94, 34.66]	33.40, 31.58, 34.90	4608
Qwen2.5-0.5B	eager	15.13%	[14.12, 16.19]	14.13, 15.23, 16.02	4608
Qwen2.5-0.5B	sdpa	9.29%	[8.48, 10.16]	9.18, 9.96, 8.72	4608
Qwen2.5-0.5B	flex_attention	9.18%	[8.38, 10.05]	10.03, 9.31, 8.20	4608

3. It reaches the objective

The clipped *fraction* of a PPO surrogate barely moves between batch shapes; *which* tokens are clipped does:

model	clipped at b1	clipped at b8	clip status flips	n
pythia-410m (NeoX 0.41B)	19.97%	20.07%	10.11%	4096
Qwen2.5-0.5B	21.41%	21.48%	1.68%	4096
Granite-3.1-1B-A400M (MoE)	3.15%	2.86%	2.78%	4096

The flip rate depends on the advantage distribution, because a token’s clip status is a joint property of its ratio and the sign of its advantage; with synthetic advantages it ranges from 8.59% (Gaussian) to 2.47% (90% zeros) on pythia-410m:

model	advantage	flips	95% CI
pythia-410m (NeoX 0.41B)	gaussian	8.59%	[7.29, 10.10]
pythia-410m (NeoX 0.41B)	binary +/-1	7.03%	[5.86, 8.42]
pythia-410m (NeoX 0.41B)	sparse (90% zero)	2.47%	[1.81, 3.38]
Qwen2.5-0.5B	gaussian	2.02%	[1.43, 2.85]
Qwen2.5-0.5B	binary +/-1	2.02%	[1.43, 2.85]
Qwen2.5-0.5B	sparse (90% zero)	0.52%	[0.26, 1.02]

4. Where it enters

Tracking the batch-1-vs-batch-8 relative divergence of the hidden states layer by layer gives the increment each layer contributes:

model	layers	Q1	Q2	Q3	Q4	after last layer	top-4 layers by increment
DeepSeek-R1-Distill-Qwen-1.5B	28	+1.1e-02	+3.7e-03	+8.6e-04	-1.1e-03	0.014	[0, 2, 3, 1]
Qwen2.5-0.5B	24	+1.1e-02	+2.1e-03	-1.3e-03	+5.0e-03	0.016	[0, 23, 22, 2]
Qwen2.5-1.5B	28	+1.0e-02	+1.4e-03	+3.9e-04	+8.3e-04	0.013	[0, 27, 1, 2]
Qwen3-1.7B	28	+1.1e-02	+4.7e-03	-1.8e-03	+1.5e-03	0.015	[0, 27, 1, 2]
pythia-410m (NeoX 0.41B)	24	+9.6e-03	+1.4e-03	+1.4e-02	+9.0e-02	0.115	[23, 19, 21, 22]

GPT-NeoX builds almost all of it in the last quarter of the stack; the Qwen2 and Qwen3 models inject it in the first five layers, carry it, and partly re-align it later. This is why an fp32 `lm_head` — the ScaleRL and MiniMax-M1 recipe, and TRL’s `cast_lm_head_to_fp32` — removes two thirds to four fifths of the out-of-band tokens on Qwen and leaves pythia essentially unchanged (34.26% $\rightarrow$ 32.24%).

5. What precision buys, and what it costs

model	bf16	fp32 head only	last 8 + head	first 8 + head	guided 8 + head	fp16, no fp32	fp16 + guided 8	all fp32 (sanity)
DeepSeek- R1- Distill- Qwen- 1.5B	6.38% (0%)	1.94% (13%)	1.33% (34%)	0.60% (34%)	0.68% (34%)	0.00% (0%)	0.00% (34%)	0.00% (87%)
Qwen2.5- 0.5B	8.92% (0%)	2.72% (22%)	1.30% (41%)	0.81% (41%)	0.86% (41%)	0.02% (0%)	0.00% (41%)	0.00% (78%)
Qwen2.5- 1.5B	8.04% (0%)	1.51% (13%)	0.94% (34%)	1.07% (34%)	0.93% (34%)	0.00% (0%)	0.00% (34%)	0.00% (87%)
Qwen3- 1.7B	5.58% (0%)	1.99% (15%)	1.42% (35%)	1.04% (35%)	1.06% (35%)	0.00% (0%)	0.00% (35%)	0.00% (85%)
pythia- 410m (NeoX 0.41B)	34.26% (0%)	32.24% (13%)	1.12% (38%)	32.31% (38%)	0.41% (38%)	0.47% (0%)	0.00% (38%)	0.00% (87%)

Holding the eight layers with the largest divergence increment in fp32, plus the head, costs 34% to 41% of the parameters and reaches 0.41% on pythia and 0.68% to 1.06% on the Qwen models; the same budget on the *last* eight layers gives 1.12% and 0.94% to 1.42%, on the *first* eight 32.31% and 0.60% to 1.07%. The profile picks the right end of the network without being told the family. It is still the wrong fix: fp16 scoring, with no fp32 anywhere, gives 0.00% on three of the four Qwen-family models, 0.02% on Qwen2.5-0.5B and 0.47% on pythia-410m, at $0.93\times$ to $0.99\times$ the bf16 time, while the guided layers cost $1.7\times$ to $2.0\times$ and all-fp32 $2.8\times$ to $3.4\times$:

model	bf16	fp16	fp32 head	last 8 + head	guided 8 + head	all fp32	peak GiB bf16 -> all fp32
Qwen2.5- 0.5B	x1.00	x0.93	x1.49	x2.05	x2.02	x3.19	7.9 -> 9.1
Qwen2.5- 1.5B	x1.00	x0.96	x1.38	x1.98	x1.94	x3.36	9.8 -> 13.2
Qwen3- 1.7B	x1.00	x0.95	x1.35	x1.93	x1.91	x3.28	10.2 -> 14.0
pythia- 410m (NeoX 0.41B)	x1.00	x0.99	x1.19	x1.71	x1.70	x2.81	3.1 -> 3.7

fp16 does not overflow where it matters here — on Qwen2.5-3B and 7B it produces no non-finite log-prob over 8,192 tokens, and the largest hidden-state magnitude it meets is 12,712 against a ceiling of 65,504:

model	fp16 non-finite log-probs	fp16 b1 vs b8	fp16 vs bf16 at b1	max abs hidden state (fp16 max 65,504)
Qwen2.5-3B	0.000%	0.05%	3.39%	3356
Qwen2.5-7B	0.000%	0.00%	3.64%	12712

This is the prescription of Precision-RL (sail-sg, arXiv:2510.26788) for the whole training loop, applied to the scoring pass alone; the layer-wise budget is the part LayerCast (arXiv:2506.09501) leaves unmeasured.

6. The other term in the ratio: truncated sampling

When an inference engine samples with `top_p`, `top_k` or `min_p`, the log probabilities it returns (vLLM’s `processed_logprobs`, which is what TRL requests) are normalised over the tokens that survived the truncation, while the trainer normalises over the vocabulary. The ratio then carries a factor equal to the kept probability mass — not a policy change:

sampling	tokens	kept mass	ratio, full vocab: mean / median / out-of-band	ratio, kept set: mean / median / out-of-band
top_p=0.8, top_k=1024	2,048	0.895	0.896 / 0.895 / 51.8%	1.001 / 1.000 / 4.3%
top_k=20	2,048	0.965	0.965 / 0.995 / 16.2%	1.000 / 1.000 / 6.2%
top_p=1.0, top_k=1024 (control)	2,048	0.994	0.995 / 1.000 / 6.9%	1.000 / 1.000 / 5.9%

vLLM 0.28 can return the kept token ids of every generated token (`return_sampling_mask`); adding the log of that mass to the engine’s log-probs removes the term and leaves the engine-vs-trainer floor of the control row. The trainer-side change is huggingface/trl pull request #7269, which keeps the trainer’s log-probs full-vocabulary — they are also the policy-gradient baseline — and lifts the engine’s instead; in a real colocated run at `top_p=0.8` the logged mismatch goes from 0.043 to 0.007 with the clip fraction unchanged.

7. Does it reach the reward?

Six ways of computing the old log-probabilities in TRL’s GRPO, run on the same seeds so that the arms share prompt order and vLLM sampling and differ only in that pass: the trainer’s own bf16 pass (A), an fp32 copy of the policy (B), bf16 with the chunk size forced to the training micro-batch (C), bf16 with the divergence-guided layers and the head in fp32 (D), bf16 with only the fp32 head (E), and an fp16 copy (F). Qwen2.5-1.5B-Instruct, GSM8K, 4 prompts $\times$ 8 completions per step, lr 2×10^{-6} , one optimisation step per generation, 150 steps, two seeds:

arm	seeds	reward, last 30 steps	reward, mean over 150	vs A, same seed	clip fraction	vLLM-vs- old abs dlogp	s/step
A default (bf16, trainer chunking)	2	0.657	0.638	-	0.0000	0.0113	9.3
B fp32 clone	2	0.658	0.641	+0.004 / -0.003	0.0005	0.0092	11.8
C bf16, chunk = micro- batch	2	0.655	0.639	-0.004 / +0.000	0.0000	0.0113	8.9
D bf16 + guided fp32 layers + head	2	0.651	0.647	+0.001 / -0.013	0.0007	0.0104	10.9
E bf16 + fp32 head only	2	0.651	0.639	-0.011 / -0.001	0.0000	0.0104	8.8

arm	seeds	reward, last 30 steps	reward, mean over 150	vs A, same seed	clip fraction	vLLM-vs- old abs dlogp	s/step
F fp16 clone	2	0.661	0.650	+0.008 / +0.001	0.0006	0.0094	8.5

None separates from the default. Per seed the last-30-step reward sits within -0.013 to $+0.008$ of arm A, against a per-step noise of sd 0.14; the clip fraction is 0.0000 to 0.0005 in every arm, because with one optimisation step per generation the ratio is 1 up to exactly this noise, and the noise never reaches the clip band. Removing the noise fully (B, F) or partly (D, E) changes nothing this experiment can see. That is the bound: about a point, with no consistent sign, on two seeds. Long-horizon collapse of the kind attributed to optimisation dynamics rather than precision (arXiv:2602.01826) is outside its reach.

8. What this harness cannot say

It is not verl’s production path: that runs FSDP2 with `use_remove_padding`, i.e. flash-attn varlen kernels, which will not build on this machine, so the one configuration that reported `max_abs_diff = 0` is the one not tested here. Everything is one GPU generation. The largest model is 7B; the production report was a 30B MoE, and within the Qwen2.5 family the rate drifts slowly with size (9.9%, 9.4%, 6.9%, 5.6% from 0.5B to 7B) while the spread across families is far larger. The clip-flip metric uses drawn, not earned, advantages, and is reported as a range for that reason. Three explanations of the author’s own were killed by the controls and are listed in the repository README: that vLLM’s batch-invariant kernels shrink it (8.0% $\rightarrow$ 7.1%, while changing trainer precision goes to 3.7% in one step), that generating under two kernel settings and comparing token by token is a valid design (45 of 64 sequences diverge), and that sequence packing removes it (32.23% with a block-diagonal mask against 31.84% with padding).

References

- sail-sg, *Defeating the Training-Inference Mismatch via FP16*, arXiv:2510.26788.
- *Beyond Precision: Training-Inference Mismatch is an Optimization Problem and Simple LR Scheduling Fixes It*, arXiv:2602.01826.
- *Give Me FP32 or Give Me Death? Challenges and Solutions for Reproducible Reasoning* (LayerCast), arXiv:2506.09501.
- ScaleRL, arXiv:2510.13786; MiniMax-M1, arXiv:2506.13585 (the fp32-head recipe).
- verl-project/verl issue #6280; huggingface/trl issue #6789 and pull request #7269; vllm-project/vllm RFC #42259 and pull request #49577 (`return_sampling_mask`).